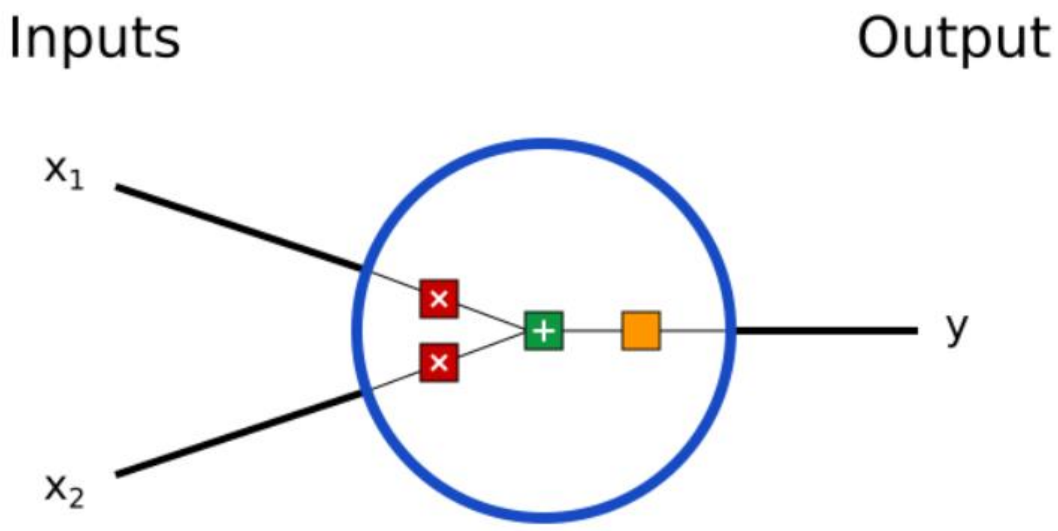
Building Blocks: Neurons

First, we have to talk about neurons, the basic unit of a neural network. A neuron takes inputs, does some math with them, and produces one output. Here's what a 2-input neuron looks like:



3 things are happening here. First, each input is multiplied by a weight:

$$x_1 \rightarrow x_1 * w_1$$

$$x_2 \rightarrow x_2 * w_2$$

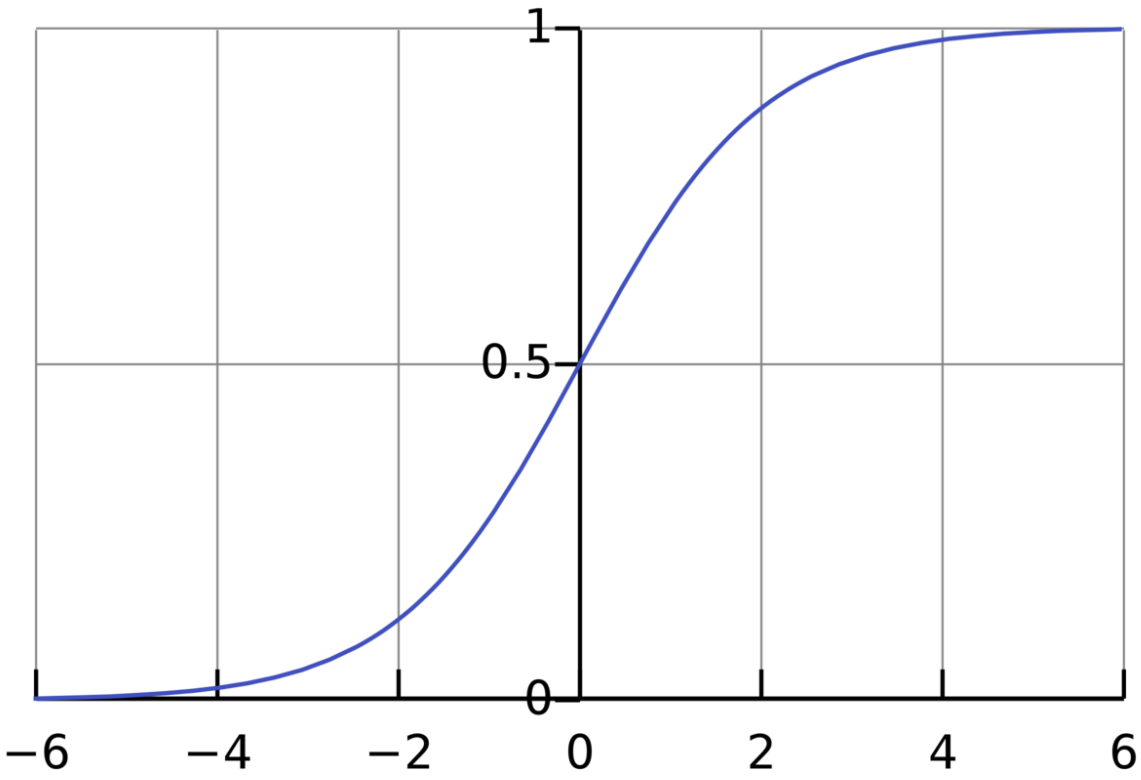
Next, all the weighted inputs are added together with a bias b:

$$(x_1 * w_1) + (x_2 * w_2) + b$$

Finally, the sum is passed through an activation function:

$$y = f(x_1 * w_1 + x_2 * w_2 + b)$$

The activation function is used to turn an unbounded input into an output that has a nice, predictable form. A commonly used activation function is the sigmoid function:



The sigmoid function only outputs numbers in the range (0,1). You can think of it as compressing $(-\infty, +\infty)$ to (0,1) — big negative numbers become ~0, and big positive numbers become ~1.

A Simple Example

Assume we have a 2-input neuron that uses the sigmoid activation function and has the following parameters:

$$w = [0, 1]$$

$$b = 4$$

w=[0, 1] is just a way of writing w1=0, w2=1 in vector form. Now, let's give the neuron an input of x=[2, 3]. We'll use the dot product to write things more concisely:

$$
\begin{aligned}
(w \cdot x) + b &= ((w_1 * x_1) + (w_2 * x_2)) + b \\
&= 0 * 2 + 1 * 3 + 4 \\
&= 7
\end{aligned}
$$

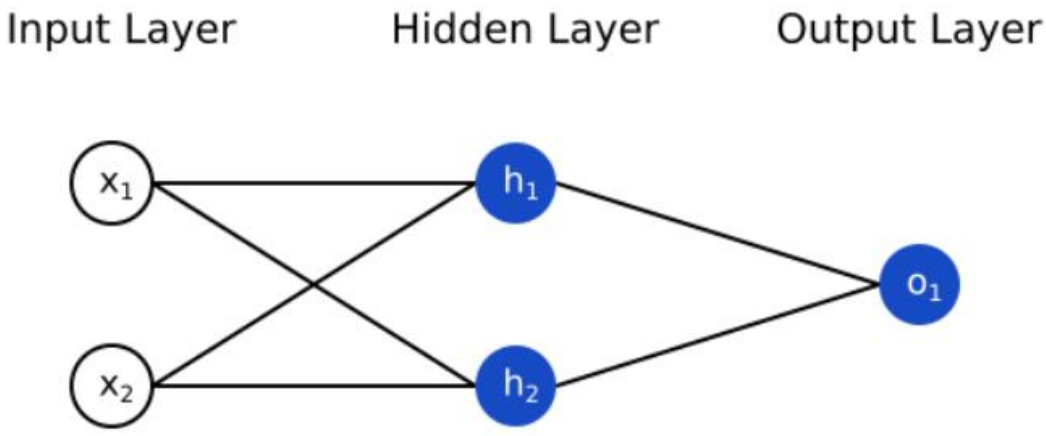
$$
y = f(w \cdot x + b) = f(7) = \boxed{0.999}
$$

The neuron outputs 0.999 given the inputs x=[2,3]. That's it! This process of passing inputs forward to get an output is known as feedforward.

Task: implement a neuron

Combining Neurons into a Neural Network

A neural network is nothing more than a bunch of neurons connected together. Here's what a simple neural network might look like:



This network has 2 inputs, a hidden layer with 2 neurons (h1 and h2), and an output layer with 1 neuron (o1). Notice that the inputs for o1 are the outputs from h1 and h2 — that's what makes this a network.

A hidden layer is any layer between the input (first) layer and output (last) layer. There can be multiple hidden layers!

An Example: Feedforward

Let's use the network pictured above and assume all neurons have the same weights w=[0,1], the same bias b=0, and the same sigmoid activation function. Let h1, h2, o1 denote the outputs of the neurons they represent.

What happens if we pass in the input x=[2, 3]?

$$\begin{aligned} h_1 = h_2 &= f(w \cdot x + b) \\ &= f((0 * 2) + (1 * 3) + 0) \\ &= f(3) \\ &= 0.9526 \end{aligned}$$

$$\begin{aligned} o_1 &= f(w \cdot [h_1, h_2] + b) \\ &= f((0 * h_1) + (1 * h_2) + 0) \\ &= f(0.9526) \\ &= \boxed{0.7216} \end{aligned}$$

The output of the neural network for input x = [2,3] is 0.7216. Pretty simple, right?

A neural network can have any number of layers with any number of neurons in those layers. The basic idea stays the same: feed the input(s) forward through the neurons in the network to get the output(s) at the end. For simplicity, we'll keep using the network pictured above for the rest of this post.

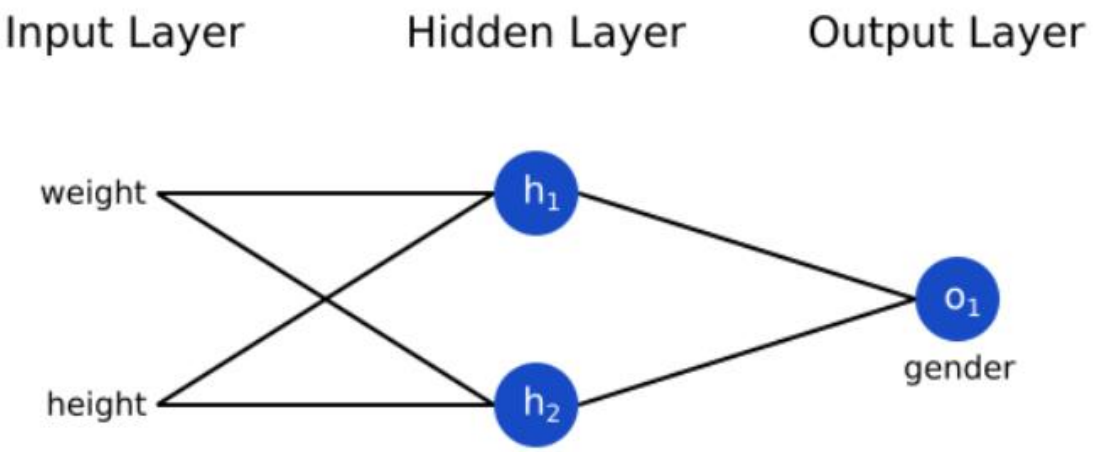
Task: implement feedforward for our neural network

Training a Neural Network:

Say we have the following measurements:

Weight	Height	Gender
133	65	F
160	75	M
152	70	M
120	60	F

Let's train our network to predict someone's gender given their weight and height:



We'll represent male with a 0 and female with a 1, and we'll also shift the data to make it easier to use:

Weight (-135)	Height (-66)	Gender
-2	-1	1
25	6	0
17	4	0
-15	-6	1

Concept of Loss Function:

Before we train our network, we first need a way to quantify how "good" it's doing so that it can try to do "better". That's what the loss is.

We'll use the mean squared error (MSE) loss:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(Y_{True} - Y_{Pred})^2$$

Let's break this down:

- n is the number of samples, which is 4 (Alice, Bob, Charlie, Diana).
- y represents the variable being predicted, which is Gender.
- y_true is the true value of the variable (the "correct answer"). For example, y_true for Alice would be 1 (Female).
- y_pred is the predicted value of the variable. It's whatever our network outputs.

$(Y_{true} - Y_{Pred})^2$ is known as the squared error. Our loss function is simply taking the average over all squared errors (hence the name mean squared error). The better our predictions are, the lower our loss will be!

Better predictions = Lower loss.

Training a network = trying to minimize its loss.

An Example Loss Calculation

Let's say our network always outputs 00 — in other words, it's confident all humans are Male. What would our loss be?

Y_{True}	Y_{Pred}	$(Y_{True} - Y_{Pred})^2$
1	0	1
0	0	0
0	0	0
1	0	1

$$MSE = \frac{1}{4}(1 + 0 + 0 + 1) = 0.5$$

Task: Implement MSE Loss Calculation